

Google Cloud 上的 Spring Native

來源：<https://codelabs.developers.google.com/spring-native-quickstart?hl=zh-tw>

步驟數：8

Spring Native 是即將在 Spring 6.x 和 Spring Boot 3.x 中進入主線的新興專案，因此現在正是熟悉這項技術的絕佳時機，可搶先在發布前幾個月做好準備。在本程式碼研究室中，開發人員將瞭解專案的價值、專案近期達成的里程碑，以及為何應將專案納入考量。

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 應用程式設定](#)
5. [5. 建構及執行原生應用程式](#)
6. [6. 部署原生應用程式](#)
7. [7. 摘要/整理](#)
8. [8. 其他資源](#)

1. 總覽

在本程式碼研究室中，我們將瞭解 Spring Native 專案、建構使用該專案的應用程式，並將其部署至 Google Cloud。

我們會介紹這個專案的元件、近期歷史記錄、一些用途，當然也會說明在專案中使用這個專案的必要步驟。

Spring Native 專案目前處於實驗階段，因此需要進行一些特定設定才能開始使用。不過，正如 SpringOne 2021 大會所宣布，Spring Native 將整合至 Spring Framework 6.0 和 Spring Boot 3.0，並提供[一流的支援](#)，因此在發布前幾個月，正是深入瞭解這個專案的絕佳時機。

雖然即時編譯已針對長時間執行的程序等項目進行最佳化，但在某些用途中，預先編譯的應用程式效能更佳，我們將在程式碼研究室中討論這點。

在接下來的研究室中

- 使用 Cloud Shell
- 啟用 Cloud Run API
- 建立及部署 Spring 原生應用程式
- 將這類應用程式部署至 Cloud Run

軟硬體需求

- 擁有 [Google Cloud Platform 專案](#)，且已啟用 GCP 帳單帳戶
- 已安裝 [gcloud CLI](#)，或可存取 [Cloud Shell](#)
- Java 和 XML 基礎技能
- 熟悉常見的 Linux 指令

問卷調查

2. 背景

Spring Native 專案運用多項技術，為開發人員提供原生應用程式效能。

如要充分瞭解 Spring Native，建議先瞭解其中幾項元件技術、這些技術為我們帶來的優勢，以及這些技術如何搭配運作。

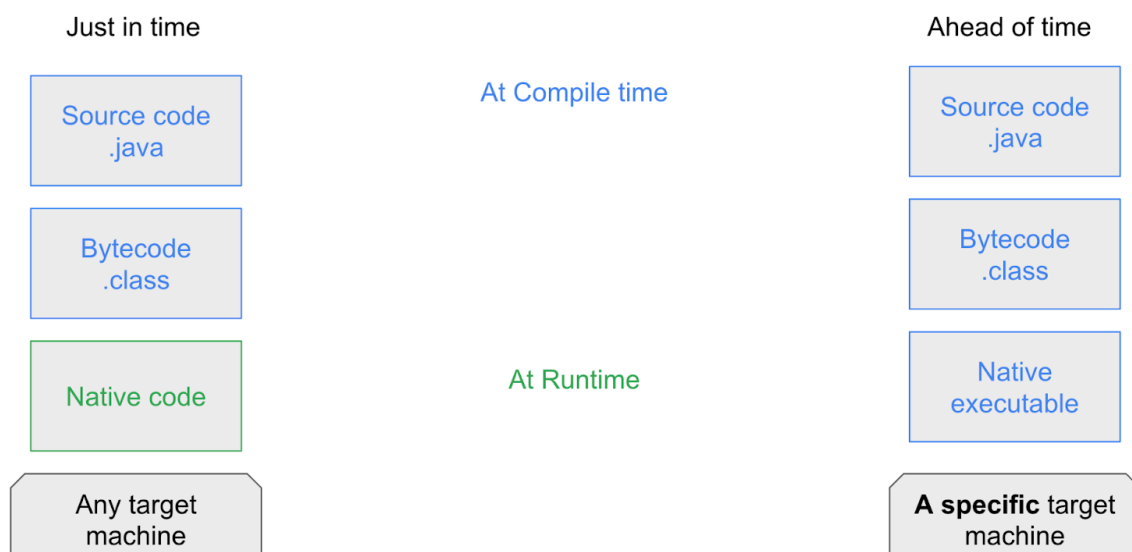
AOT 編譯

開發人員在編譯時正常執行 `javac` 時，我們的 `.java` 原始碼會編譯成以位元碼編寫的 `.class` 檔案。這個位元碼只能由 Java 虛擬機器解讀，因此 JVM 必須在其他機器上解讀這段程式碼，我們才能執行程式碼。

這個程序賦予 Java 簽章可攜性，讓我們「編寫一次，隨處執行」，但與執行原生程式碼相比，成本較高。

幸好，大多數 JVM 實作項目都使用即時編譯，可降低這項解譯成本。具體做法是計算函式的叫用次數，如果叫用次數夠多，超過閾值：門檻（預設為 10,000 次），系統就會在執行階段將函式編譯為原生程式碼，避免後續進行耗費資源的解譯作業。

預先編譯則採取相反的做法，在編譯時將所有可到達的程式碼編譯為原生可執行檔。這項做法會犧牲可攜性，換取執行階段的記憶體效率和其他效能提升。



當然，這是一種取捨，不一定值得。不過，AOT 編譯在某些用途中非常實用，例如：

- 啟動時間很重要的短期應用程式
- 記憶體受限的環境，JIT 可能會耗用過多資源

有趣的是，AOT 編譯是在 JDK 9 中以實驗功能的形式推出，但這項實作的維護成本高昂，且從未普及，因此在 Java 17 中悄悄移除，改為讓開發人員使用 GraalVM。

GraalVM

GraalVM 是經過高度最佳化的開放原始碼 JDK 發行版本，具有極快的啟動時間、AOT 原生映像檔編譯功能，以及多語言功能，可讓開發人員將多種語言混合到單一應用程式中。

GraalVM 仍在積極開發中，不斷推出新功能並改良現有功能，因此建議開發人員密切關注。

近期里程碑包括：

- 全新且容易使用的原生圖片建構輸出內容 (2021 年 1 月 18 日)

- 支援 Java 17 ([2022 年 1 月 18 日](#))
- 預設啟用多層級編譯，縮短多語言編譯時間 ([2021 年 4 月 20 日](#))

Spring Native

簡單來說，Spring Native 可讓您使用 GraalVM 的原生映像檔編譯器，將 Spring 應用程式轉換為原生可執行檔。

這項程序包括在編譯時間對應用程式執行靜態分析，找出應用程式中可從進入點存取的所有方法。

這基本上會建立應用程式的「封閉世界」概念，也就是假設所有程式碼在編譯時都已知，且不允許在執行階段載入新程式碼。

請注意，產生原生映像檔是記憶體密集型程序，所需時間比編譯一般應用程式更長，且會對 Java 的某些方面造成[限制](#)。

在某些情況下，應用程式不需要變更任何程式碼，就能與 Spring Native 搭配運作。不過，在某些情況下，您必須進行特定的原生設定，才能正常運作。在這種情況下，Spring Native 通常會提供[原生提示](#)，簡化這個程序。

步驟 3 · 約 8 分鐘

3. 設定/準備工作

開始導入 Spring Native 之前，我們需要先建立及部署應用程式，建立效能基準，以便稍後與原生版本比較。

1. 建立專案

首先，請從 start.spring.io 取得應用程式：

```
curl https://start.spring.io/starter.zip -d dependencies=web \  
      -d javaVersion=11 \  
      -d bootVersion=2.6.4 -o io-native-starter.zip
```

這個範例應用程式使用 Spring Boot 2.6.4，這是撰寫本文時 [spring-native 專案支援的最新版本](#)。

請注意，自 [GraalVM 21.0.3](#) 發布後，您也可以使用 Java 17 執行這個範例。在本教學課程中，我們仍會使用 Java 11，盡量減少相關設定。

在指令列中取得 ZIP 檔案後，我們可以為專案建立子目錄，並將資料夾解壓縮至該處：

```
mkdir spring-native  
  
cd spring-native  
  
unzip ../io-native-starter.zip
```

2. 程式碼變更

開啟專案後，我們會快速新增生命跡象，並在執行專案時展示 Spring Native 效能。

編輯 `DemoApplication.java`，使其符合下列內容：

```

package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.event.ApplicationReadyEvent;
import org.springframework.context.event.EventListener;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.Duration;
import java.time.Instant;

@RestController
@SpringBootApplication
public class DemoApplication {
    private static Instant startTime;
    private static Instant readyTime;

    public static void main(String[] args) {
        startTime = Instant.now();
        SpringApplication.run(DemoApplication.class, args);
    }

    @GetMapping("/")
    public String index() {
        return "Time between start and ApplicationReadyEvent: "
            + Duration.between(startTime, readyTime).toMillis()
            + "ms";
    }

    @EventListener(ApplicationReadyEvent.class)
    public void ready() {
        readyTime = Instant.now();
    }
}

```

此時，我們的基準應用程式已準備就緒，因此請隨意建構映像檔並在本機執行，以便在轉換為原生應用程式之前，瞭解啟動時間。

建構映像檔：

```
mvn spring-boot:build-image
```

您也可以使用 `docker images demo` 瞭解基準圖片的大小：

```

aaronmwanjala@cloudshell:~/spring-native (video-test-run-two)$ docker images demo
REPOSITORY    TAG                IMAGE ID           CREATED           SIZE
demo          0.0.1-SNAPSHOT    c25a674479ef      42 years ago     259MB

```

如要執行應用程式，請按照下列步驟操作：

```
docker run --rm -p 8080:8080 demo:0.0.1-SNAPSHOT
```

3. 部署基準應用程式

現在我們有了應用程式，接下來要部署應用程式並記錄時間，稍後會與原生應用程式的啟動時間進行比較。

視您建構的應用程式類型而定，有幾種不同的「您的內容」[代管](#)方式。

不過，由於我們的範例是非常簡單的網頁應用程式，因此可以簡化作業，只使用 Cloud Run。

如果您是在自己的電腦上操作，請務必安裝並更新 [gcloud CLI](#) 工具。

如果您使用 Cloud Shell，系統會處理所有事項，您只需在來源目錄中執行下列指令：

```
gcloud run deploy
```

4. 應用程式設定

1. 設定 Maven 存放區

由於這個專案仍處於實驗階段，我們必須設定應用程式，才能找到 Maven 中央存放區沒有的實驗構件。

這項作業需要在 pom.xml 中加入下列元素，您可以在所選的編輯器中完成這項作業。

在 pom 中新增下列 repositories 和 pluginRepositories 區段：

```
<repositories>
  <repository>
    <id>spring-release</id>
    <name>Spring release</name>
    <url>https://repo.spring.io/release</url>
  </repository>
</repositories>

<pluginRepositories>
  <pluginRepository>
    <id>spring-release</id>
    <name>Spring release</name>
    <url>https://repo.spring.io/release</url>
  </pluginRepository>
</pluginRepositories>
```

2. 新增依附元件

接著，新增 spring-native 依附元件，這是以原生映像檔執行 Spring 應用程式的必要條件。注意：[如果您使用 Gradle，則不需要執行這個步驟](#)

```
<dependencies>
  <!-- ... -->
  <dependency>
    <groupId>org.springframework.experimental</groupId>
    <artifactId>spring-native</artifactId>
    <version>0.11.2</version>
  </dependency>
</dependencies>
```

3. 新增/啟用外掛程式

現在新增 AOT 外掛程式，改善原生映像檔的相容性和占用空間 ([瞭解詳情](#))：

```

<plugins>
  <!-- ... -->
  <plugin>
    <groupId>org.springframework.experimental</groupId>
    <artifactId>spring-aot-maven-plugin</artifactId>
    <version>0.11.2</version>
    <executions>
      <execution>
        <id>generate</id>
        <goals>
          <goal>generate</goal>
        </goals>
      </execution>
    </executions>
  </plugin>
</plugins>

```

現在，我們要更新 `spring-boot-maven-plugin`，啟用原生映像檔支援功能，並使用 `paketo` 建構工具建構原生映像檔：

```

<plugins>
  <!-- ... -->
  <plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
      <image>
        <builder>paketo/buildpacks/builder:tiny</builder>
        <env>
          <BP_NATIVE_IMAGE>true</BP_NATIVE_IMAGE>
        </env>
      </image>
    </configuration>
  </plugin>
</plugins>

```

請注意，[微型建構工具](#) 圖片只是其中一個選項。這個映像檔的額外程式庫和公用程式非常少，有助於縮小攻擊面，因此很適合我們的用途。

舉例來說，如果您要建構的應用程式需要存取某些常見的 C 程式庫，或您還不確定應用程式的需求，[完整建構工具](#) 可能更適合您。

5. 建構及執行原生應用程式

完成上述步驟後，我們應該就能建構映像檔，並執行編譯後的原生應用程式。

執行建構作業前，請留意下列事項：

- 這項作業所需時間會比一般建構作業（幾分鐘）長

```
[INFO] Successfully built image 'docker.io/library/demo:0.0.1-SNAPSHOT'
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 06:02 min
```

- 這個建構程序可能會佔用大量記憶體（幾 GB）

```
[INFO] [creator] 67,443 (63.92%) of 105,506 methods reachable
[INFO] [creator] 658 classes, 167 fields, and 3,062 methods registered for reflection
[INFO] [creator] 68 classes, 88 fields, and 55 methods registered for JNI access
[INFO] [creator] [3/7] Building universe... (9.1s @ 4.34GB)
[INFO] [creator] [4/7] Parsing methods... [***] (8.2s @ 2.85GB)
[INFO] [creator] [5/7] Inlining methods... [*****] (15.3s @ 1.96GB)
[INFO] [creator] [6/7] Compiling methods... [*****] (79.5s @ 3.07GB)
[INFO] [creator] [7/7] Creating image... (8.2s @ 2.21GB)
[INFO] [creator] 27.19MB (40.88%) for code area: 43,837 compilation units
[INFO] [creator] 32.91MB (49.48%) for image heap: 9,317 classes and 408,432 objects
[INFO] [creator] 6.42MB ( 9.65%) for other data
[INFO] [creator] 66.52MB in total
[INFO] [creator] -----
[INFO] [creator] Top 10 packages in code area: Top 10 object types in image heap:
[INFO] [creator] 1.59MB sun.security.ssl 12.45MB byte[] for general heap data
[INFO] [creator] 1004.08KB java.util 3.27MB java.lang.Class
[INFO] [creator] 865.47KB com.oracle.svm.core.reflect 2.76MB java.lang.String
[INFO] [creator] 673.60KB com.sun.crypto.provider 2.33MB byte[] for java.lang.String
[INFO] [creator] 608.94KB org.apache.tomcat.util.net 1.37MB java.util.LinkedHashMap
[INFO] [creator] 529.44KB org.apache.catalina.core 1007.65KB java.lang.reflect.Method
[INFO] [creator] 495.82KB sun.security.x509 592.22KB java.util.HashMap$Node
[INFO] [creator] 475.88KB org.apache.coyote.http2 530.16KB java.lang.String[]
[INFO] [creator] 429.12KB java.util.concurrent 510.47KB c.o.s.c.u.LazyFinalReference
[INFO] [creator] 389.07KB java.lang 440.19KB java.util.HashMap$Node[]
[INFO] [creator] ... 555 additional packages ... 2283 additional object types
[INFO] [creator] (use GraalVM Dashboard to see all)
[INFO] [creator] -----
[INFO] [creator] 29.9s (10.1% of total time) in 41 GCs | Peak RSS: 6.63GB | CPU load: 3.43
[INFO] [creator] -----
```

- 這個建構程序需要 Docker Daemon 可連線
- 雖然在本範例中，我們是手動完成這項程序，但您也可以設定建構階段，[自動觸發原生建構設定檔](#)。

建構映像檔：

```
mvn spring-boot:build-image
```

建構完成後，我們就可以查看原生應用程式的實際運作情形！

如要執行應用程式，請按照下列步驟操作：

```
docker run --rm -p 8080:8080 demo:0.0.1-SNAPSHOT
```

此時，我們已準備好瞭解原生應用程式方程式的兩端。

我們在編譯時間耗費了一些時間，也使用了額外的記憶體用量，但換來的是啟動速度更快、耗用記憶體明顯較少的應用程式（視工作負載而定）。

如果我們執行 `docker images demo` 來比較原生圖片與原始圖片的大小，會發現圖片大幅縮小：

```
aaronmwanjala@cloudshell:~/spring-native (video-test-run-two)$ docker images demo
REPOSITORY    TAG                IMAGE ID           CREATED           SIZE
demo          0.0.1-SNAPSHOT    29601e765073      42 years ago     88.7MB
```

此外，在較複雜的用途中，您還需要進行額外修改，向 AOT 編譯器說明應用程式在執行階段的行為。因此，某些可預測的工作負載 (例如批次工作) 可能非常適合這項功能，而其他工作負載則可能需要更多時間。

步驟 6 · 約 6 分鐘

6. 部署原生應用程式

如要將應用程式部署到 Cloud Run，我們需要將原生映像檔放入套件管理工具，例如 [Artifact Registry](#)。

1. 準備 Docker 存放區

我們可以先建立存放區，開始這個程序：

```
gcloud artifacts repositories create native-image-docker-repo --repository-format=docker \
--location=us-central1 --description="Repository for our native images"
```

接下來，我們要驗證是否已通過驗證，可將內容推送至新登錄檔。

gcloud CLI 可大幅簡化該程序：

```
gcloud auth configure-docker us-central1-docker.pkg.dev
```

2. 將映像檔推送至 Artifact Registry

接著，我們會標記圖片：

```
export PROJECT_ID=$(gcloud config list --format 'value(core.project)')

docker tag demo:0.0.1-SNAPSHOT \
us-central1-docker.pkg.dev/$PROJECT_ID/native-image-docker-repo/quickstart-image:tag2
```

然後使用 `docker push` 將其傳送至 Artifact Registry：

```
docker push us-central1-docker.pkg.dev/$PROJECT_ID/native-image-docker-repo/quickstart-
image:tag2
```

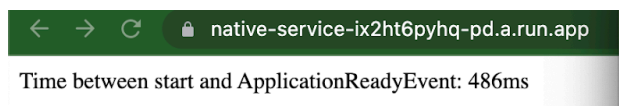
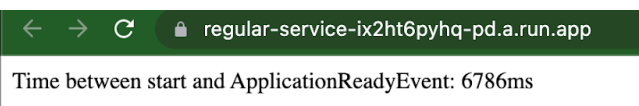
3. 部署至 Cloud Run

現在，我們已準備好將儲存在 Artifact Registry 中的映像檔部署至 Cloud Run：

```
gcloud run deploy --image us-central1-docker.pkg.dev/$PROJECT_ID/native-image-docker-
repo/quickstart-image:tag2
```

由於我們已將應用程式建構及部署為原生映像檔，因此可以放心，應用程式在執行時會充分利用基礎架構成本。

歡迎自行比較基準應用程式與這個新原生應用程式的啟動時間！

	
Time between start and ApplicationReadyEvent: 486ms	Time between start and ApplicationReadyEvent: 6786ms

步驟 7 · 約 4 分鐘

7. 摘要/整理

恭喜您在 Google Cloud 建構及部署 Spring 原生應用程式！

希望本教學課程能幫助您進一步瞭解 Spring Native 專案，並在日後有需要時採用。

選用：清理及/或停用服務

無論您是為這個程式碼研究室建立 Google Cloud 雲端專案，還是重複使用現有專案，請務必避免因我們使用的資源產生不必要的費用。

您可以[刪除或停用](#)我們建立的 Cloud Run 服務、[刪除代管的映像檔](#)，或[關閉](#)整個專案。

步驟 8 · 約 3 分鐘

8. 其他資源

雖然 Spring Native 專案目前仍處於實驗階段，但已有許多實用資源，可協助早期採用者排解問題及參與專案：

其他資源

以下是可能與本教學課程相關的線上資源：

- [進一步瞭解原生提示](#)
- [進一步瞭解 GraalVM](#)
- [如何參與](#)
- [建構原生映像檔時發生記憶體不足錯誤](#)
- [應用程式無法啟動錯誤](#)

授權

這項內容採用的授權為 創用 CC 姓名標示 2.0 通用授權。
